

Contents

guide Ray: A Distributed Framework for Emerging AI Applications	1
0. 这篇 guide 怎么读	1
1. 当时的历史语境	2
2. 原论文的 story	2
3. 论文结构地图	3
4. 核心概念先打底	3
5. 一张总图: Ray 把 RL 循环变成动态任务图	4
6. API 层: 为什么 task 和 actor 必须同时存在	4
7. Computation graph: 三种 edge	5
8. 架构层: application layer 和 system layer	6
9. GCS: 论文里最重要的系统设计	7
10. Bottom-up scheduler: 先本地, 再全局	7
11. Object store: 为什么对象必须不可变	8
12. End-to-end: add.remote(a, b) 到 ray.get(c)	8
13. Fault tolerance: task replay 和 actor checkpoint	8
14. 本仓库的最小代码对应	9
15. 数学和资源形状	10
16. 实验证据链条	11
17. 这篇论文没有证明什么	12
18. 现代意义	12
19. 本仓库学习路径	13
20. 常见误读	13
21. 用 AI agent 学这篇论文	14
22. 闭卷掌握检查	14

guide_Ray: A Distributed Framework for Emerging AI Applications

原论文: Ray: A Distributed Framework for Emerging AI Applications

本地原文 PDF: [learning/training-orchestration/paper/01_ray.pdf](#)

本地导读 PDF: [learning/training-orchestration/paper/guide_01_ray.pdf](#)

作者: Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, Ion Stoica

机构: UC Berkeley

版本: arXiv v2, 2018-09-30, 初版 2017

0. 这篇 guide 怎么读

Ray 这篇论文不是在教你 “怎么 pip install ray”, 而是在回答一个系统问题:

如果一个 AI 应用同时需要模拟环境、训练模型、在线推理、动态产生新任务、处理 CPU/GPU 异构资源、并且要在失败后恢复, 运行时系统应该长什么样?

读完这篇 guide, 你应该能闭卷画出 Ray 的四个核心部件:

- task 和 actor 统一在动态计算图里。
- object store 保存任务输入输出, 并把数据位置登记到 GCS。

- GCS 保存控制状态、对象目录、任务 lineage、函数表、事件日志。
- bottom-up distributed scheduler 先在本地调度，必要时交给全局调度器做资源和数据 locality 权衡。

这篇论文的难点不是公式，而是“系统设计为什么这样分层”。如果你只记住“Ray 有 task 和 actor”，还没有读懂论文。真正的核心是：Ray 把控制面、数据面、调度面拆开，让动态细粒度任务能扩到百万级每秒，同时还能保留 fault tolerance。

1. 当时的历史语境

Ray 发表时，分布式系统和机器学习系统已经有很多成熟工具，但它们各自擅长的 workload 不一样。

Big Data 系统像 MapReduce、Spark、Dryad 适合批处理或比较规则的数据流。它们通常有 stage、barrier、集中调度、粗粒度任务这些特征。对日志分析和 ETL 很有效，但不适合“毫秒级、不规则、动态生成”的模拟任务。

深度学习系统像 TensorFlow、MXNet、PyTorch 当时主要围绕神经网络训练。它们能高效使用 GPU/TPU，能表达静态或半静态计算图，适合 supervised learning 的大 batch 训练，但不天然支持仿真、策略服务、任务动态分支和 actor 状态。

Serving 系统像 TensorFlow Serving、Clippier 负责在线模型推理。它们关心延迟、吞吐、模型管理和 A/B 测试，但不负责分布式训练和模拟。

Task-parallel 系统像 CIEL、Dask 能表达动态任务图和 futures，但当时对长生命周期 stateful computation、RL 训练里的 parameter server、GPU 上的迭代计算、embedded serving 支持不足。

Actor 系统像 Akka、Orleans 擅长 stateful actor，但不强调不可变对象、任务 lineage、数据 locality 和 stateless task recovery。

Ray 的目标就是夹在这些系统之间：

- 像 Dask/CIEL 一样表达动态 task graph。
- 像 actor framework 一样保留 stateful actor。
- 像 ML runtime 一样理解 CPU/GPU 异构资源。
- 像 dataflow system 一样用 lineage 做恢复。
- 像高性能系统一样把 scheduler 和 object metadata 从中心瓶颈里拆开。

2. 原论文的 story

论文从强化学习 RL 切入，不是因为 Ray 只能做 RL，而是因为 RL 把系统压力集中暴露出来。

一个典型 RL 应用会反复做三件事：

- Simulation: 用当前 policy 在环境里 rollout，产生 trajectory。
- Training: 用 trajectory 更新 policy。
- Serving: 在环境交互时，把 state 输入 policy，返回 action。

这三件事在 supervised learning 里可以相对分离：离线训练一套系统，在线推理另一套系统。但 RL 的循环是紧耦合的：simulator 需要最新 policy，policy update 需要 simulator 刚产生的数据，policy serving 又在 simulator 内部被频繁调用。

论文把这个循环抽象成一个系统需求：

- computation duration 异构：有的任务 5ms，有的任务几小时。
- resource 异构：simulation 常用 CPU，training 常用 GPU，serving 可能两者都用。
- computation graph 动态：某个 rollout 结果会决定后面要不要继续模拟。

- state 形态混合: 有些任务是 stateless 的, 有些组件必须保留模型权重或 simulator state。
- 吞吐要求极高: 论文给了一个简单估算, 200 台机器, 每台 32 核, 每个 task 5ms, 就需要 1,280,000 tasks/s 才能吃满集群。

所以 Ray 的主张不是 “写 Python 更方便”。它真正的主张是:

对 emerging AI applications, 必须用一个统一执行引擎同时表达 task-parallel computation 和 actor-based computation, 并且用分布式控制状态、分布式调度、分布式对象存储支撑它。

3. 论文结构地图

建议你按下面顺序读原文 PDF:

- Abstract: 直接看三件事, 统一 task/actor, 分布式 scheduler, GCS, 超过 1.8M tasks/s。
- Section 1 Introduction: 读 Ray 为什么不是 Spark、TensorFlow、Clipper、Dask 的简单替代。
- Section 2 Motivation and Requirements: 读 RL 的 simulation/training/serving 循环和 1.28M tasks/s 的估算。
- Section 3 Programming and Computation Model: 读 task、actor、future、ray.get、ray.wait、data edge、control edge、stateful edge。
- Section 4 Architecture: 这是全篇最重要部分, 慢读 GCS、bottom-up scheduler、object store、Figure 7 end-to-end flow。
- Section 5 Evaluation: 读 Figure 8 到 Figure 14, 理解每个实验到底证明哪个系统设计。
- Section 7 Discussion: 读 API 演化、limitations、fault tolerance 是否有必要、GCS 的调试价值。
- Section 8 Conclusion: 回到论文主张, 确认你能把证据链串起来。

4. 核心概念先打底

Task

Ray 的 task 是 stateless remote function execution。调用 `f.remote(args)` 会立刻返回 future。task 的输出只依赖输入对象, 因此可以被重新执行。这一点很关键: 只有 task 接近 side-effect-free, lineage-based recovery 才成立。

Actor

Actor 是 stateful computation。`Class.remote()` 创建远程 actor, `actor.method.remote()` 调用方法。actor 方法串行执行, 并且共享 actor 内部状态。它适合 parameter server、GPU trainer、第三方 simulator 这种 “状态不容易序列化或每次初始化很贵” 的组件。

Future/ObjectRef

Ray API 返回 future。future 可以作为另一个 remote function 的参数传递, 不需要先 get。这让用户用普通 Python 写出数据依赖, 运行时再根据 future 的 readiness 触发任务。

ray.get 和 ray.wait

`ray.get` 等待一个或多个 future 全部完成并取回结果。`ray.wait` 等待前 k 个完成的 future, 适合 rollout 时长不均匀的 simulation。RL 里 simulator 有快有慢, 如果总是等齐, 就会回到 BSP barrier 的低利用率问题。

Immutable Object Store

每个 task 的输入输出存在 per-node object store。对象不可变, 因此不用做复杂一致性协议。对象位置和大小登记到 GCS, 调度器可以根据 object locality 做 placement。

GCS, Global Control Store

GCS 是 Ray 的控制状态中心。它不是普通数据库的配角，而是论文的系统设计核心。GCS 存对象表、任务表、函数表、事件日志、lineage，并用 sharding 和 chain replication 保证扩展和容错。

Bottom-up Scheduler

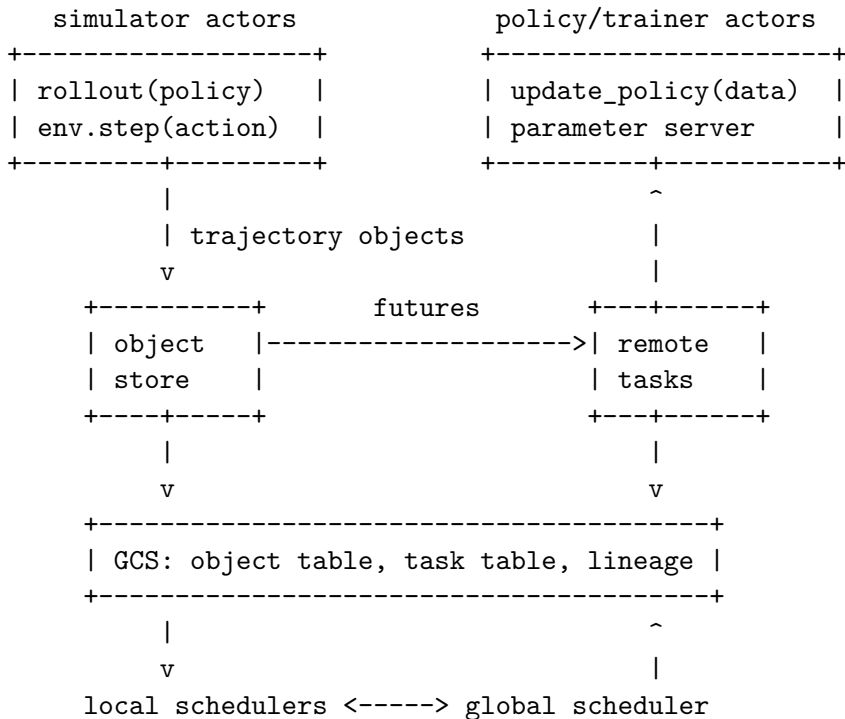
Ray 不是所有任务都先进中心调度器。任务先到本地 scheduler；如果本地队列不长且资源满足，就本地跑。只有本地过载或资源不满足时，才交给 global scheduler。global scheduler 再用 node load、resource constraints、remote input transfer cost 做决策。

Lineage

Lineage 记录“哪个任务生成了哪个对象，以及任务依赖哪些对象”。如果对象丢了，可以重新执行生成它的任务。actor 的 stateful edge 也进入 graph，这样 actor 方法输出也能有恢复路径。

5. 一张总图: Ray 把 RL 循环变成动态任务图

RL application



图里有一个容易忽略的点: Ray 不是把所有东西都塞进 scheduler。object location 不放在 scheduler 私有状态里，而是放到 GCS。这样数据拷贝、对象目录、任务选择可以解耦，scheduler 不必在每个 object transfer 上变成瓶颈。

6. API 层: 为什么 task 和 actor 必须同时存在

Ray 的 API 很小:

```
@ray.remote
def f(x):
    return x + 1
```

```

future = f.remote(41)
value = ray.get(future)

@ray.remote
class Trainer:
    def step(self, batch):
        ...

trainer = Trainer.remote()
loss_future = trainer.step.remote(batch)
ready, pending = ray.wait([loss_future], num_returns=1)

```

论文的关键不是 API 多漂亮，而是 API 背后对应两种不同系统属性。

Task 的优点:

- 可以细粒度 load balance。
- 可以根据输入对象位置移动 computation。
- stateless，因此恢复成本低。
- 适合大量 simulation、preprocess、postprocess、动态搜索。

Task 的缺点:

- 对频繁更新的小状态不划算，因为状态要作为对象读写。
- 对 GPU 模型权重这类大状态反复序列化不划算。

Actor 的优点:

- 状态常驻，适合 trainer、parameter server、simulator wrapper。
- 细粒度 method call 可以直接更新内部状态。
- 可以把模型权重、环境句柄、GPU context 留在进程里。

Actor 的缺点:

- 一旦 actor 放到某个节点，不容易为了一个大输入对象随意迁移。
- stateful failure recovery 更难，需要 stateful edge 和 checkpoint。
- 负载均衡粒度比 stateless task 更粗。

Ray 的统一点在于: actor method invocation 也被放进动态计算图里。它像 task 一样有 future 和依赖，但额外带有 actor state 的顺序约束。

7. Computation graph: 三种 edge

论文里 Figure 4 的真正信息是三种 edge。

Data edge:

```

task T produces object D
object D is input to task U

```

```
T ---> D ---> U
```

Control edge:

```
task T dynamically invokes task V
```

```
T ---> V
```

Stateful edge:

actor method M1 happens before method M2 on the same actor

Actor A:

M1 ---> M2 ---> M3

用中文说:

- data edge 表示对象依赖。
- control edge 表示嵌套 remote call 产生的动态依赖。
- stateful edge 表示 actor 内部状态的时间顺序。

为什么 stateful edge 重要? 因为 actor 方法看起来只是 “调用一个对象的方法”, 但它其实依赖上一次方法执行后的内部状态。如果不把这个依赖写进 graph, 系统恢复时就不知道 actor state 应该从哪里 replay。

8. 架构层: application layer 和 system layer

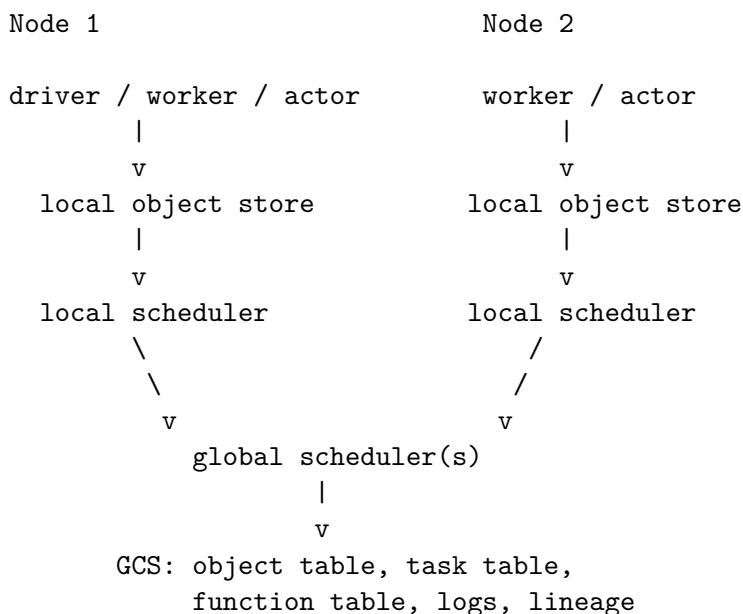
Ray 的架构分两层。

Application layer 有三类进程:

- Driver: 执行用户程序, 提交 remote function 或 actor method。
- Worker: stateless process, 串行执行 task, 不保留跨 task 状态。
- Actor: stateful process, 串行执行暴露的方法, 保留内部状态。

System layer 有三大组件:

- GCS: 保存控制状态。
- Distributed scheduler: local schedulers 加 global scheduler。
- Distributed object store: 每个节点一个 object store, 用 shared memory 支持同节点 zero-copy。



这张图要看出两个分离:

- 控制面 metadata 在 GCS。
- 数据面 object 在 object store 之间复制。

如果 object metadata 和 scheduler 紧耦合，那么每次 allreduce 或大对象转移都要经过 scheduler，scheduler 会被放到 latency critical path 上。Ray 选择把 object table 放进 GCS，让调度决策和 object transfer 分开。

9. GCS: 论文里最重要的系统设计

GCS 的基本形态是 sharded key-value store 加 pub-sub。每个 shard 用 chain replication 容错。它保存：

- Object Table: object id 到 object location、size、readiness。
- Task Table: task id 到输入、输出、状态、lineage。
- Function Table: remote function 的注册信息。
- Event Logs: debug、profiling、timeline。

GCS 解决三个问题。

第一，fault tolerance。任务输出丢了，系统需要知道哪个 task 能重新生成它。这个 lineage 不能只放在某个 worker 或 scheduler 私有内存里，因为节点会挂。

第二，scalability。细粒度动态任务每秒百万级，单 master 存所有 lineage 会变成瓶颈。GCS 用 sharding 扩展，scheduler 和 object store 都可以横向扩。

第三，debuggability。把控制状态集中登记，调试工具和 Web UI 可以查询整个系统状态。论文讨论里特别提到，GCS 对调试 Ray 自身和应用 timeline 都很有用。

一个新手误区是把 GCS 理解成“中央调度器”。不对。GCS 是 control state storage，不是所有逻辑执行的中心。Ray 的调度逻辑可以有多个 global scheduler replica，共享 GCS 状态。

10. Bottom-up scheduler: 先本地，再全局

调度目标不是“全局最优”，而是在极高任务吞吐下做足够好的资源和 locality 决策。

本地调度器的策略：

- task 在某个节点生成后，先提交给该节点 local scheduler。
- 如果本地队列不长，且资源满足，就本地执行。
- 如果本地过载，或者缺少 GPU/CPU 等资源，就转给 global scheduler。

全局调度器的策略：

- 筛掉资源不满足的节点。
- 对每个候选节点估计等待时间。
- 加上远程输入对象复制的时间。
- 选择估计完成最快的节点。

可以写成下面的简化代价：

```
score(node, task)
    = queue_wait_ms(node)
    + remote_input_mb(task, node) / bandwidth_mb_per_ms
```

choose node with minimal score among resource-feasible nodes

这个公式背后的设计理由很朴素：对小对象，队列负载更重要；对大对象，数据 locality 更重要；对 GPU task，资源约束先于 locality。

11. Object store: 为什么对象必须不可变

Ray 的 object store 是每个节点一个 in-memory store，任务输入输出放在里面。同节点 worker 通过 shared memory 读取对象，论文实现里使用 Apache Arrow 作为数据格式。

对象不可变的好处：

- 不需要复杂一致性协议。
- 多个 task 可以安全共享同一个对象。
- 失败时可以通过 lineage 重算对象。
- object location 只是“哪里有副本”，不需要追踪写冲突。

对象传输流程：

task needs object X

local scheduler checks local object store

|
| missing
v

object store asks GCS for X locations

|
v

copy X from remote object store

|
v

task starts after all inputs local

论文也明确说明了限制：object store 不支持分布式对象，每个对象要能放进单个节点。大矩阵、大树这类 distributed object 可以在应用层拆成一组 futures。

12. End-to-end: add.remote(a, b) 到 ray.get(c)

论文 Figure 7 用 add(a, b) 展示一次完整路径。把它翻译成中文流程：

1. add remote function 注册到 GCS 的 Function Table。
2. Driver 调用 add.remote(a, b)。
3. 本地 scheduler 先接收 task，必要时转给 global scheduler。
4. global scheduler 查 GCS，知道 a 在 N1，b 在 N2。
5. scheduler 结合 locality 和负载，决定把 task 放在某个节点。
6. 目标节点 local scheduler 检查本地 object store。
7. 如果缺输入对象，就从远端 object store 复制。
8. worker 通过 shared memory 读输入对象，执行 add。
9. 输出 c 写入本地 object store。
10. GCS Object Table 登记 c 的位置。
11. Driver 调用 ray.get(c) 时，如果本地没有 c，先在 GCS 注册 callback。
12. c ready 后，复制到 driver 所在节点并返回给用户程序。

关键观察：控制面请求很多，但多数任务本地调度，GCS 查询会缓存。论文承认单个示例看起来 RPC 多，但真实负载里不会每一步都走慢路径。

13. Fault tolerance: task replay 和 actor checkpoint

Ray 的 task recovery 靠 lineage。

如果 object D 丢了，系统查到 D 是 task T 的输出，再递归检查 T 的输入对象是否还在。如果输入也丢了，就继续回溯。最后按依赖顺序 replay 任务。

```
O1 = task_a()
O2 = task_b(O1)
O3 = task_c(O2)
```

```
If O3 is lost:
    replay task_a if O1 missing
    replay task_b if O2 missing
    replay task_c
```

Actor 更麻烦。因为 actor 的输出不只依赖显式输入，还依赖内部状态。Ray 用 stateful edges 把 actor method calls 串起来，再允许用户定义 checkpoint，避免从 actor 创建开始全部 replay。

Actor trainer

```
step_0 -> step_1 -> checkpoint -> step_2 -> step_3
```

```
If actor fails after step_3:
    restore checkpoint
    replay step_2 and step_3
```

论文在讨论部分还解释了为什么 AI 系统需要 fault tolerance。即使 RL 有统计噪声，不能简单说“丢几个 rollout 无所谓”。容错让程序更容易写、错误更容易复现、也能用更便宜的 spot instances。

14. 本仓库的最小代码对应

这次我补了一个文件：

- learning/training-orchestration/src/ray_original_minimal.py

它不依赖真实 Ray，而是把论文机制压成可测试 toy model：

- GlobalControlStore: 保存 object table、task table、function table、actor stateful edge。
- NodeState: 保存 CPU/GPU resource 和 queue delay。
- choose_node_bottom_up: 本地优先，过载后用 queue wait 加 remote input transfer cost 选择节点。
- submit_task: 记录 task lineage，生成 output object。
- actor_method_call: 把 actor method 写成带 stateful edge 的 task。
- reconstruct_lineage: 从丢失对象回溯需要 replay 的 task。
- ray_wait: 模拟等待前 k 个 ready futures。

最小示例：

```
gcs = GlobalControlStore()
big = gcs.put_object("big-batch", size_mb=1000.0, locations={"B"})

nodes = {
    "A": NodeState("A", cpus_total=8, gpus_total=0, queued_ms=200.0),
    "B": NodeState("B", cpus_total=8, gpus_total=0, queued_ms=0.0),
}

task = TaskSpec("t1", "preprocess", input_refs=[big])
```

```
chosen = choose_node_bottom_up(task, "A", nodes, gcs)

# A is overloaded and B owns the large input object, so B wins.
assert chosen == "B"
```

这个 toy example 对应论文 Figure 8a 的 intuition: 如果调度不看数据位置, 大对象输入会把 task latency 放大很多; 如果调度看 locality, 大对象不用反复跨节点搬。

15. 数学和资源形状

Ray 论文没有复杂 loss function, 但有几个系统公式或估算必须会。

任务吞吐估算

```
tasks_per_second
    = n_nodes * cores_per_node * (1000 / task_duration_ms)
```

Example:

```
200 nodes * 32 cores/node * (1000 / 5ms)
= 1,280,000 tasks/s
```

这个估算不是 benchmark, 而是需求推导。它说明如果任务粒度小到毫秒级, 调度系统必须扛住百万级吞吐, 否则 CPU 资源会被调度开销饿住。

调度代价估算

```
score(node)
    = queue_wait_ms(node)
    + remote_input_mb(node) / bandwidth_mb_per_ms
```

这里的变量含义:

- queue_wait_ms(node): 节点已有排队任务造成的等待。
- remote_input_mb(node): task 需要但该节点本地没有的输入对象总大小。
- bandwidth_mb_per_ms: 平均网络传输带宽。

这个公式说明 scheduler 同时考虑 compute queue 和 data movement。它不是精确最优解, 因为 runtime 不知道完整未来 graph, 也只用简单平均估计执行时间和带宽。

对象和任务的资源形状

```
TaskSpec:
    input_refs: [object_id_1, object_id_2, ...]
    output_refs: [object_id_out]
    cpus: integer
    gpus: integer
    duration_ms: estimated or observed average
```

```
ObjectRecord:
    size_mb: float
    locations: set[node_id]
    creator_task: task_id
    ready: bool
```

```
NodeState:
```

```
cpu_free, gpu_free
queued_ms
local_object_ids
```

这就是系统论文里的“张量形状”。对 serving/infra 类论文，不要只盯神经网络 tensor，要把 request、object、node、queue、bandwidth 当成一等对象。

16. 实验证据链条

Ray 的实验不是一个总表解决所有问题，而是把系统 claim 拆成多个问题验证。

Evidence 1: locality-aware scheduling

Figure 8a 比较 task placement 是否感知 object locality。实验让 1000 个 task 带随机对象依赖，在两节点上调度。locality-aware policy 的 latency 基本不随对象大小增长；不感知 locality 的策略在 10MB 到 100MB 输入时 latency 增加 1 到 2 个数量级。

这证明的不是“Ray 比某系统快”，而是“把 object metadata 放进 GCS，并让 scheduler 使用 object location，确实能避免大对象远程搬运的灾难”。

Evidence 2: end-to-end task throughput

Figure 8b 用 empty tasks 测 Ray 横向扩展。Ray 在 60 nodes 超过 1M tasks/s，在 100 nodes 超过 1.8M tasks/s，100M tasks 用约 54 秒处理完。

这个实验证明 GCS 加 bottom-up scheduler 的架构能支撑极端细粒度 task。它不证明所有真实应用都能线性扩展，因为真实应用可能有对象依赖、同步瓶颈、应用层串行部分。

Evidence 3: object store performance

Figure 9 报告单客户端 object store 写入吞吐和小对象 IOPS。大对象超过 15GB/s，小对象约 18K IOPS。这个实验支撑 shared-memory object store 的性能基础，也解释为什么 object store 能作为 task/actor 之间的数据交换层。

Evidence 4: GCS fault tolerance and flushing

Figure 10a 模拟 GCS chain replication reconfiguration，客户端观察到的最大延迟低于 30ms。Figure 10b 表明如果不 flush，50M no-op tasks 的 lineage/control metadata 会吃满内存；启用 flushing 后内存可以被限制住。

这说明 GCS 不是“无限记账本”，必须有 garbage collection 或 flushing。论文把这点作为系统限制和工程方向。

Evidence 5: task and actor failure recovery

Figure 11a 用线性 task chain 和节点移除展示 task reconstruction。节点移除时会重算丢失依赖，节点加回后吞吐恢复。Figure 11b 展示 actor reconstruction: 10 个节点里杀掉 2 个，2000 个 actors 里的 400 个要恢复。checkpoint 后只需 replay 约 500 次方法调用；如果没有 checkpoint，会接近 10K 次 replay。

这证明 stateful edge 加 checkpoint 能把 actor recovery 放进统一 lineage 思路里。

Evidence 6: allreduce

Figure 12a 把 Ray API 实现的 ring allreduce 和 OpenMPI 比较。在 16 nodes、100MB 和 1GB 对象上，Ray 分别约 200ms 和 1200ms，论文报告对 OpenMPI 有 1.5x 和 2x 的优势。原因不是 Ray 天然比 MPI 强，而是该环境里 Ray 多线程网络传输更充分利用 25Gbps 网络。小对象时 OpenMPI 仍然更强，因为 MPI 有低开销算法。

Figure 12b 给 scheduler ablation: 给 Ray ring reduce 人工加几毫秒调度延迟, completion time 就会接近翻倍。这直接支撑 “scheduler 不能上关键路径” 的设计。

Evidence 7: building blocks

Distributed training: Figure 13 用 TensorFlow ResNet-101 比 Ray+TF、Horovod+TF、Distributed TF。Ray 的 parameter-server SGD 能接近 Horovod, 并在 10% 内接近 Distributed TF。这里证明 Ray 的通用 API 可以表达专用训练系统里的关键优化, 比如 gradient computation、transfer、summation 的 pipeline。

Serving: Table 3 比 Clipper 和 Ray 在 embedded serving 场景下的吞吐。小输入 Ray 约 6200 states/s, Clipper 约 4400 states/s; 较大输入 Ray 约 6900 states/s, Clipper 约 290 states/s。这个实验的适用范围很窄: client/server colocated, 服务嵌在 RL dynamic graph 里。它不证明 Ray 替代 Clipper 的生产模型管理能力。

Simulation: Table 4 比 MPI bulk synchronous 和 Ray asynchronous tasks。256 CPUs 上, MPI 约 2.16M timesteps/s, Ray 约 4.03M timesteps/s。证据点是 ray.wait 和异步收集能减少 barrier 造成的空转。

Evidence 8: RL applications

Evolution Strategies: Ray 版本扩到 8192 cores, median time 约 3.7 minutes, 超过当时 best published result 约 10 minutes; special-purpose reference 在更高 core 数失败。关键优化是 actor aggregation tree, 而 Ray 的 nested tasks/actors 让这种结构容易表达。

PPO: Ray 版本在 Humanoid-v1 上超过高度优化 MPI 实现, 同时使用更少 GPU。原因是 Ray 能表达异构资源: rollout 可以放 CPU-only 实例, policy update 用 GPU。论文还估算 CPU-only 资源和 spot fault tolerance 组合能显著降成本。

17. 这篇论文没有证明什么

第一, Ray 不是 Spark 的全功能替代。论文明确说 Ray 没有 Spark 丰富的 data-parallel API、straggler mitigation、query optimization。

第二, Ray 不是 Clipper 或 TensorFlow Serving 的全功能替代。Ray 强调 embedded serving, 不负责模型生命周期管理、线上测试、模型组合等 serving 系统完整功能。

第三, Ray 的 object model 有限制。对象不可变、单对象放单节点, 这让一致性和 recovery 简化, 但也把 distributed object 的复杂性推到应用层。

第四, scheduler 的估计不是全局最优。Ray 不能预知完整未来 graph, 只能用 queue size、平均执行时间、平均带宽做启发式选择。

第五, lineage 不是免费。每个 task 都要记录 lineage, GCS 需要 flushing 或 GC, 否则长时间运行会有 metadata 成本。

第六, actor recovery 仍需要用户参与。用户定义 checkpoint 才能避免长链 replay; 否则 stateful computation 的恢复成本可能很高。

18. 现代意义

今天 Ray 已经远超论文最初的 RL 语境, 变成 Python 分布式 AI 的重要生态之一。你会在这些地方看到这篇论文的影子:

- LLM training orchestration 里的 actor、worker、placement group、resource scheduling。
- RLHF/RLAIF pipeline 里的 rollout workers、reward model workers、trainer actor。

- LLM serving 和 batch inference 里的 object store、distributed futures、任务编排。
- Agent 系统里的并行 tool execution、异步任务、long-running stateful worker。
- AutoML/hyperparameter tuning 里的 dynamic task graph 和 fault-tolerant execution。

但更重要的是系统品味: Ray 告诉你, 一个通用 runtime 要同时追求易用、灵活、吞吐、延迟、容错, 必须拆清楚:

- 哪些状态是 control state。
- 哪些数据是 immutable object。
- 哪些逻辑应该在 scheduler。
- 哪些逻辑应该在 worker/actor。
- 哪些东西需要 lineage。
- 哪些东西必须靠 checkpoint。

这对你学习 LLM infra 非常关键。很多训练和推理系统的真正难点, 不是单个模型公式, 而是资源、状态、数据移动和失败恢复如何一起工作。

19. 本仓库学习路径

相关 lecture:

- `learning/training-orchestration/lectures/04-ray.md`

相关代码:

- `learning/training-orchestration/src/ray_actors.py`
- `learning/training-orchestration/src/ray_original_minimal.py`
- `learning/training-orchestration/src/fault_tolerance.py`
- `learning/training-orchestration/src/elastic_training.py`
- `learning/training-orchestration/src/capstone_cluster_run.py`

建议 30 到 60 分钟实验:

1. 运行本专题测试:

```
.venv\Scripts\python.exe learning\training-orchestration\src\tests\test_all.py
```

2. 打开 `ray_original_minimal.py`, 把 `queue_threshold_ms` 从 50 改成 500。

3. 观察 `choose_node_bottom_up` 的本地优先行为如何改变。

4. 把 big object 的 `size_mb` 从 1000 改成 1。

5. 观察 data locality 的重要性怎么下降。

6. 给 `actor_method_call` 加 checkpoint 概念, 模拟只 replay checkpoint 之后的 actor methods。

预期观察:

- 本地队列轻时, Ray 宁可本地跑, 减少 global scheduler 压力。
- 本地队列很重且远端有大对象时, global scheduler 会偏向对象所在节点。
- 输入对象越大, locality 越重要。
- actor method 的 `stateful_dep` 形成一条链, 恢复时不能随意跳过。

20. 常见误读

误读 1: Ray 就是一个任务队列。

不对。普通任务队列通常不保存完整计算图 lineage, 也不把 object store、actor stateful edges、resource scheduling 统一起来。

误读 2: Actor 比 task 更高级, 所以应该都用 actor。

不对。actor 保留状态, 但牺牲了 task 的灵活迁移和低成本 recovery。大对象 postprocess、simulation fan-out、动态搜索更适合 task。

误读 3: GCS 是瓶颈, 所以集中控制状态一定错误。

不对。Ray 的设计是“逻辑集中, 物理分片, 组件无状态”。GCS 存共享 metadata, 不等于所有调度逻辑集中在单进程。

误读 4: Ray allreduce 快, 所以 Ray 替代 MPI。

不对。论文只说明在特定对象大小和 AWS 网络条件下, Ray 的 API 可以表达高性能 allreduce。小对象和专用通信场景仍可能是 MPI/NCCL 更合适。

误读 5: RL 可以丢 rollout, 所以 fault tolerance 不重要。

不对。容错不仅是保留统计样本, 还关系到程序可推理、错误可复现、便宜资源可用。

21. 用 AI agent 学这篇论文

你可以让 agent 帮你, 但要让它逼你把概念落到代码和图上。推荐这样用:

我正在读 Ray 论文。请你一次只问一个问题, 按下面顺序考我:

1. 为什么 RL 的 simulation/training/serving 会逼出 Ray 的系统需求?
2. task 和 actor 的系统属性差别是什么?
3. GCS 保存哪些控制状态, 为什么不能只放 scheduler 里?
4. bottom-up scheduler 的 cost model 怎么写?
5. Figure 8 到 Figure 14 每个实验分别证明哪个设计?
6. 请让我把答案对应到 ray_original_minimal.py 里的函数。

我回答后, 请指出不严谨处, 并要求我补一张 ASCII 图。

你还可以让 agent 做“反向审稿人”:

请站在系统论文 reviewer 的角度, 指出 Ray 论文证据链的强处和弱处。

不要泛泛评价。每个观点必须对应一个 figure、table 或 section。

然后给我一个本仓库能跑的 toy ablation。

22. 闭卷掌握检查

读完后请闭卷回答:

1. Ray 为什么从 RL 讲起? RL 的哪三个 workload 必须紧耦合?
2. task 和 actor 在 load balancing、data locality、fault tolerance、state update 成本上分别有什么取舍?
3. 解释 data edge、control edge、stateful edge, 并画出一个 actor method chain。
4. GCS 存哪些表? 为什么它是 control state storage, 而不是普通 centralized scheduler?
5. 写出 Ray global scheduler 的简化 score, 并解释对象大小如何影响 placement。
6. object store 为什么使用 immutable objects? 这带来什么限制?
7. add.remote(a, b) 到 ray.get(c) 的端到端流程有哪些控制面和数据面步骤?
8. Figure 8a、8b 分别证明什么? 不能证明什么?
9. Figure 11 的 task reconstruction 和 actor reconstruction 有什么不同?
10. 为什么 Figure 12b 能支持“scheduler latency critical”的结论?

11. Ray 为什么在 embedded serving 上能比 Clipper 快? 这个结论为什么不能外推到所有 serving 场景?
12. 如果让你在本仓库扩展 `ray_original_minimal.py`, 你会怎么加 checkpoint, 怎么测试 replay 次数下降?

真正掌握的标志是: 你能不用原文, 把 Ray 画成 “API -> computation graph -> GCS/object store/scheduler -> experiments” 的因果链, 并用本仓库代码解释链条里的每个箭头。